MySQL 读写性能测试

Spring Data JPA 与 MyBatis

实验日期：2026 年 10 月 18 日

摘要

本实验使用 JMeter 测试 Spring 应用的商品读取与新建接口，完成 JPA 性能测试、MyBatis 读写效率比较及两种实现的对比。负载参数参考实验报告样例，读取设 5 档线程数、写入设 4 档，两种实现共 18 组、31200 次请求，业务断言均通过。使用 Prometheus 记录 JVM 活跃线程、CPU 和内存使用率。

关键词：MySQL；Spring Data JPA；MyBatis；JMeter

1 问题描述

在相同应用、数据库和接口下切换 JPA 与 MyBatis，记录读写请求的响应时间，比较不同线程档位下的测试结果。材料包括原始 JTL、JMeter 报告、监控数据和过程截图，供验收时逐项核对。

2 实验分析

2.1 实验环境

服务器	用途	主要软件
CDWY3	编译与压力测试	JDK 21、Maven 3.9.16、JMeter 5.6.3
CDWY1	应用服务	Spring Boot 4.1.1、JPA、MyBatis 3.5.19
CDWY2	数据库	MySQL 9.2.0
CDWY4	监控服务	Prometheus 3.7.3

四台服务器均为 Ubuntu 18.04.6、2 核 CPU、2 GiB 内存，通过私网连接。应用使用 Docker Swarm 部署，各节点运行 node-exporter。初始数据库含商品 3912 行、关联 30660 行、上架 3914 行。每组测试前恢复相同初始数据。

应用 Tomcat 最大/最小线程 300/100，Druid 最大/最小/初始连接 300/100/100。应用堆 256–512 MiB，JMeter 堆 256–768 MiB，线程栈 512 KiB。JPA 和 MyBatis 使用同一应用镜像，通过 profile 切换；两套实现各 16 项功能测试通过。

2.2 实验方法与读取结果

读取接口为 GET /shops/{shopId}/products/{id}，商品 ID 来自 CSV；写入接口为 POST /shops/1/products，字段按 JMX 计划生成。读取断言 HTTP 200 且 errno=0，写入断言 HTTP 201。JMeter 以非 GUI 模式运行，保存 JTL 并生成 HTML 报告。

操作	线程数	升压	循环	两实现合计
读取	1000、1050、1100、1150、1200	10 秒	2 次	10 组 22000 请求
写入	1000、1100、1200、1300	10 秒	1 次	8 组 9200 请求

本报告统一使用已采集的第一轮，所有配置均按相同规则保留，不选择性更换轮次。线程数表示启动的线程总数，循环次数表示每个线程执行请求的次数。例如 1000-10-2 表示 1000 线程在 10 秒内启动，每线程请求 2 次，共 2000 次请求。

平均响应时间和 P95 由原始 elapsed 计算；P95 表示约 95%的请求耗时不超过该值。参考样例，以至少 80%的请求成功且 elapsed≤40 ms 作为比较条件。原始数据的请求数、失败数、均值及吞吐量已与 JMeter 报告核对。

ReadProduct 结果

线程数	实现	请求数	均值 ms	P95 ms	≤40 ms 比例	达到 80%
1000	JPA	2000	9.69	44.05	94.75%	是
1000	MyBatis	2000	3.67	6.00	98.60%	是
1050	JPA	2100	2.72	4.00	99.52%	是
1050	MyBatis	2100	2.01	3.00	100.00%	是
1100	JPA	2200	3.18	5.00	99.41%	是
1100	MyBatis	2200	2.34	4.00	100.00%	是
1150	JPA	2300	2.26	3.00	99.96%	是
1150	MyBatis	2300	2.15	3.00	99.61%	是
1200	JPA	2400	3.86	7.00	98.88%	是
1200	MyBatis	2400	2.56	4.00	99.54%	是

10 组读取请求全部成功，均达到 80%的比较条件。MyBatis 平均响应为 2.01–3.67 ms，JPA 为 2.26–9.69 ms。1200 线程时，两者分别为 2.56 ms 和 3.86 ms；各 2400 次请求。

2.3 读取响应分布与时间趋势

每行对应一档线程数，左右两图使用同一组请求。左图显示完整响应时间的累计百分比；右图按请求发起时间分成 1 秒区间，显示扣除 JMeter 记录的连接建立耗时后的平均响应，即 max(elapsed−Connect, 0)。

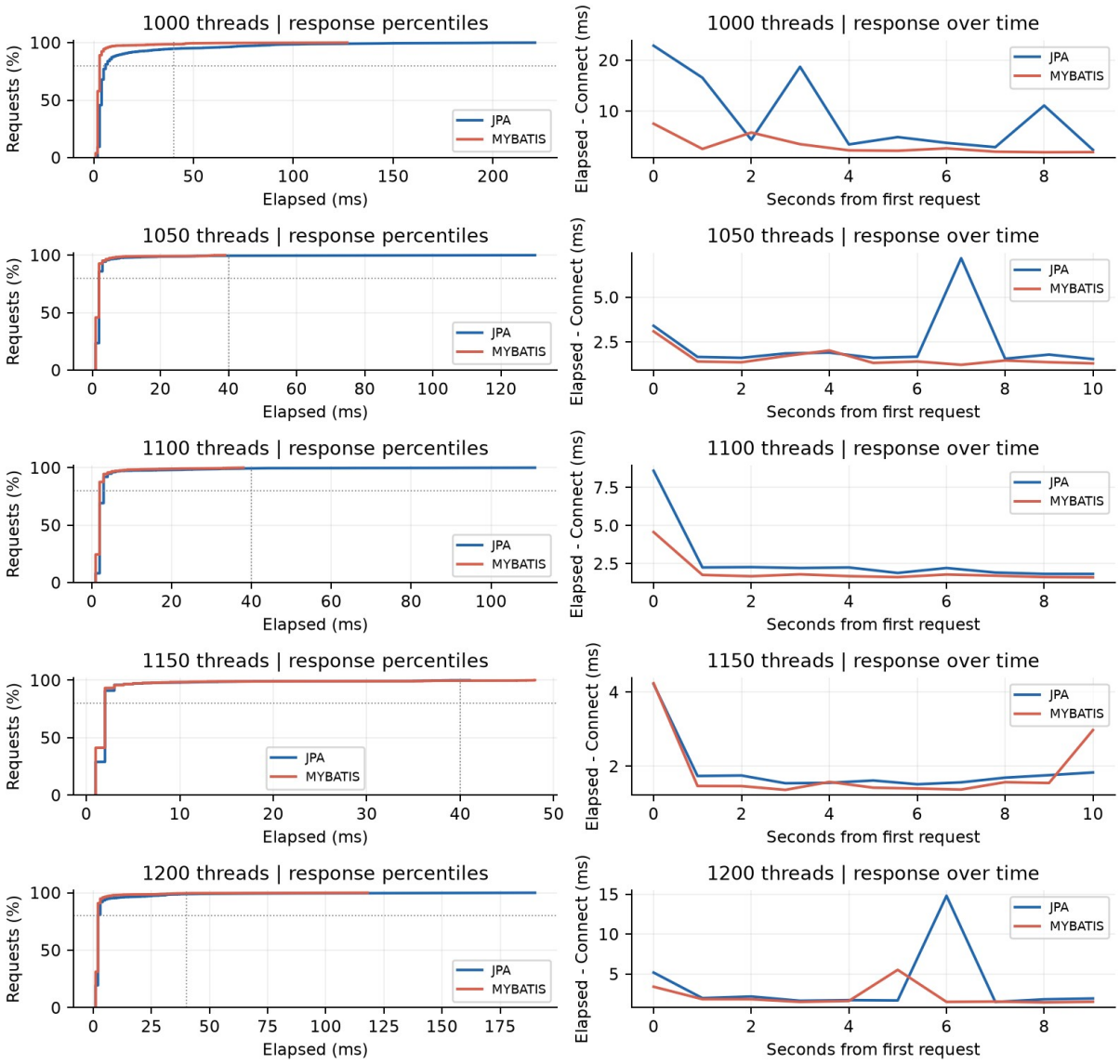


图 1 JPA 与 MyBatis 读取测试，10 秒升压、2 次循环。

百分比图中的虚线为 40 ms 和 80%；时间趋势图从该组首个请求开始计时。图表数据均来自本报告对应的原始 JTL。

2.4 写入结果与读写比较

写入每线程执行 1 次请求，四档分别产生 1000、1100、1200 和 1300 次请求。8 组断言全部通过，写后新增商品数量均与成功请求数一致。

线程数	实现	请求数	均值 ms	P95 ms	≤40 ms 比例	达到 80%
1000	JPA	1000	7.28	8.00	98.80%	是
1000	MyBatis	1000	6.16	7.00	98.90%	是
1100	JPA	1100	16.43	66.05	93.45%	是
1100	MyBatis	1100	7.30	10.00	98.64%	是
1200	JPA	1200	8.13	11.00	98.42%	是
1200	MyBatis	1200	6.18	7.00	99.08%	是
1300	JPA	1300	3232.80	4163.10	0.00%	否
1300	MyBatis	1300	102.03	615.30	71.77%	否

两种实现的 1000、1100、1200 线程写入均达到 80%的比较条件。1300 线程时，MyBatis 平均响应为 102.03 ms，40 ms 内成功比例为 71.77%；JPA 平均响应为 3232.80 ms，40 ms 内成功比例为 0%，两者该组均未达到比较条件。

MyBatis 读写比较

线程数	读取均值 ms	写入均值 ms	读取/写入循环
1000	3.67	6.16	2 次 / 1 次
1100	2.34	7.30	2 次 / 1 次
1200	2.56	6.18	2 次 / 1 次

在样例中读写共有的 1000、1100、1200 线程档位下，MyBatis 的读取平均耗时均小于写入。读取与写入的循环次数按样例分别设置为 2 次和 1 次。

JPA 与 MyBatis 比较

同一操作和线程档位下，两种实现使用相同的升压时间、循环次数、请求生成规则和初始数据，仅切换持久层。在本次保留的各档读取和写入记录中，MyBatis 平均响应时间均低于 JPA；具体数值见读取表和写入表。

2.5 写入响应分布与时间趋势

写入曲线的统计方式与读取一致。1300 线程档位中，两种实现出现较长响应时间，表现为百分比曲线向右延伸，时间趋势图中也可看到对应的耗时变化。

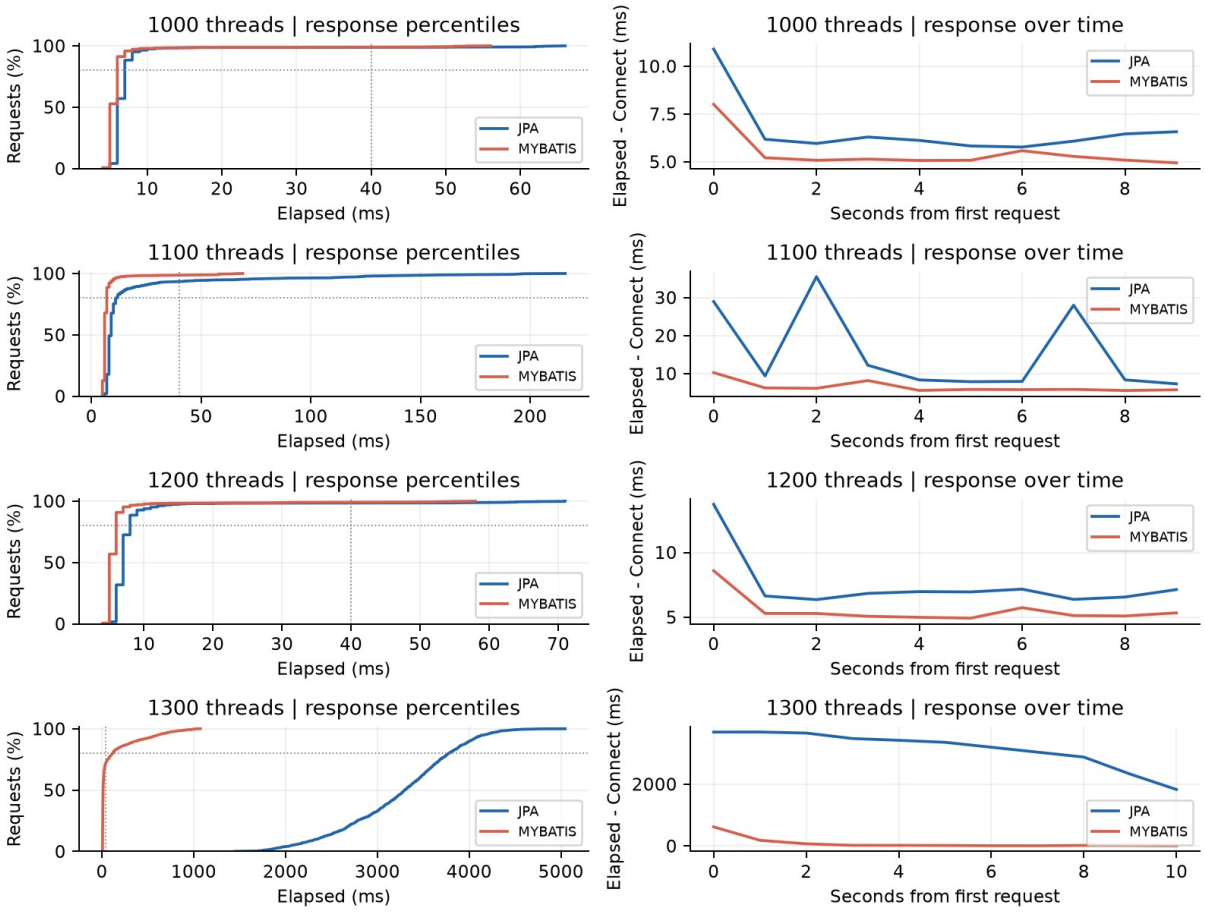


图 2 JPA 与 MyBatis 写入测试，10 秒升压、1 次循环。

每组原始 JTL 保存全部请求，包括超过 40 ms 的请求。业务断言成功表示接口返回符合测试计划，达到响应时间条件则由 elapsed 另行判断。

2.6 运行监控

按照实验三要求，查询 spring:jvm_threads_live_threads、node:cpu_usage_percent 和 node:memory_usage_percent，导出各组时间窗口内的监控数据。下图展示 1300 线程写入时的应用监控，分别对齐两组首个请求。

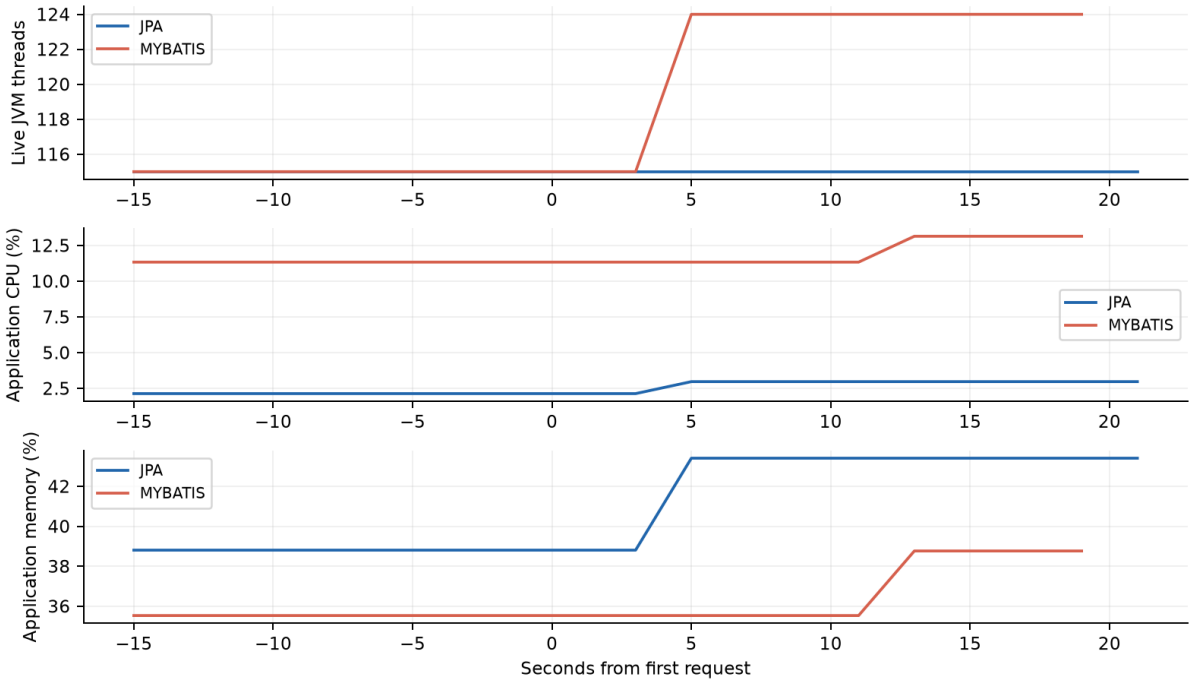


图 3 1300 线程写入的 JVM 活跃线程、应用节点 CPU 与内存使用率。

JVM 活跃线程表示应用进程中的存活线程数。CPU 使用率按配置中的 rate[5m]规则计算；内存使用率为[1−(MemFree+Buffers+Cached)/MemTotal]×100%。监控抓取间隔 2 秒，记录规则组每 30 秒计算一次。

3 总结

本实验完成 JPA 读写性能测试、MyBatis 读写比较及两种实现的对比。所提交 18 组共 31200 次请求，断言失败为 0；10 组读取及 6 组写入达到参考样例的响应时间比较条件，两个 1300 线程写入组未达到。原始请求、监控记录、JMeter 报告和测试截图均已保存。

程序地址：https://github.com/Re1lya/lab3-mysql-benchmark

参考资料

[1] 实验三：MySQL 读写性能测试，第 1–4 页。

[2] 实验报告样例，第 1–10 页。负载档位与报告结构据此整理，测量数据来自本组实验。